CAN-Based Multi-Node ESP32 System for Fault Detection and Autonomous Task Reallocation to a Backup Node with IoT Dashboard Monitoring

Phase - 1 Project Report

Submitted by

Sarathkumar B (411725401001)

in partial fulfillment for the award of the

degree of

MASTERS OF

ENGINEERING IN

APPLIED ELECTRONICS

PRINCE SHRI VENKATESHWARA PADMAVATHY ENGINEERING COLLEGE

[AN AUTONOMOUS INSTITUTION],

PONMAR, CHENNAI-127

ANNA UNIVERSITY: CHENNAI - 600025

OCTERBER 2026

BONAFIDE CERTIFICATE

Certified that this Phase-1 Project report **"CAN-Based Multi-Node ESP32 System for Fault Detection and Autonomous Task Reallocation to a Backup Node with IoT Dashboard Monitoring"** is the bonafide work of **"Sarathkumar B (411725401001)"** who carried out the project under my supervision.

SIGNATURE

Dr. K.K. SENTHILKUMAR, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR
Department of ECE,
Prince Shri Venkateshwara Padmavathy Engineering College, Ponmar,
Chennai- 600 127.

SIGNATURE

G.VIJAYALAKSHMI , M.E.,

PROJECT COORDINATOR

ASSISTANT PROFESSOR
Department of ECE,
Prince Shri Venkateshwara Padmavathy EngineeringCollege,
Ponmar, Chennai- 600 127.

Submitted for the project Viva-Voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our deep gratitude to our honorable Chairman **Dr. K. VASUDEVAN M.A., M.Phil., Ph.D**. for rending the technical staffs in completing this project.

We convey our sincere gratitude to our honorable Vice Chairman, **Dr. V. VISHNU KARTHIK, M.D.,MBBS** for his keen interest in our studies and the facilities offered in the premier institution.

We thank our Dean **Dr. V. MAHALAKSHMI, ME., Ph.D**., for her valuable guidance and encouragement in all our efforts throughout this course.

We thank our Principal **Dr. G. INDIRA, M.E., Ph.D**., for her valuable guidance and encouragement in all our efforts throughout this course.

We convey our sincere gratitude to our honorable Head of the Department, coordinator, **Dr. K.K. SENTHILKUMAR, M.E., Ph.D.,** for complete support and constant encouragement without whom this project would not achieved fruition.

We would thank our coordinator, **G.VIJAYALAKSHMI, M.E., ASSISTANT PROFESSOR** for her constant, encouragement and technical advice.

We also express our deep gratitude to all other faculty members and Lab assistants for their moral support rendered to us.

We could like to express our sincere gratitude to all our family members and friends for their loving support and encouragement.

ABSTRACT

The proposed project presents a CAN communication-based multi-node ESP32 system for fault detection and autonomous task reallocation to a backup node with IoT dashboard monitoring. The system consists of a Master ESP32 and four sensor nodes interconnected through a Controller Area Network (CAN) bus. Node 1 monitors temperature and humidity using a DHT11 sensor, Node 2 detects vibration, and Node 3 performs motion monitoring using an MPU6050 sensor. Node 4 is designed as a universal backup node equipped with a DHT11 sensor, a vibration sensor, and an MPU6050 sensor to support the monitoring tasks of the primary nodes.

The Master ESP32 continuously monitors node health through heartbeat messages and identifies failures based on predefined timeout conditions. When a primary node becomes unavailable, the Master initiates task reassignment to the corresponding backup sensor on Node 4. This mechanism aims to reduce monitoring interruptions and improve system reliability. Sensor readings, node status, CAN communication information, fault events, and backup activation details are transmitted from the Master ESP32 to a cloud database using Wi-Fi and HTTP/HTTPS communication. A React.js web dashboard integrated with Supabase provides centralized visualization of sensor data, node health, fault detection, and recovery events.

The proposed system combines distributed embedded communication, autonomous fault management, sensor redundancy, and cloud-based monitoring in a unified architecture. The prototype is intended to demonstrate the feasibility of dynamic backup task allocation in embedded sensor networks.

Keywords: ESP32, CAN Bus, Fault Detection, Task Reallocation, Universal Backup Node, Embedded Systems, IoT Dashboard, Supabase.

TABLE OF CONTENTS

CHAPTER NO	TABLE OF CONTENTS	PAGE NO
	ABSTRACT	III
	TABLE OF CONTENTS	IV
	LIST OF FIGURE	VI
	LIST OF ABBREVIATIONS AND SYMBOLS	VII
1	1.1 Introduction	1
	1.2 Objective	2
2	Literature Survey	4
3	3.1 Working Principle	8
	3.2 System Design	10
	3.3 PCB Design Of Power Section	11
4	4.1 Hardware And Software Implementation	14
	4.1.1 ESP32 Development Board	14
	4.1.2 SN65HVD230 CAN Transceiver Module	16
	4.1.3 DHT11 Sensor	16
	4.1.4 Vibration Sensor	17
	4.1.5 MPU6050 Accelerometer and Gyroscope	18
	4.1.6 OLED Display	18
	4.1.7 Two-Channel Relay Module	19
	4.1.8 Power Supply and Interconnection	19
	4.2 Software Implementation	20
	4.2.1 Embedded Software Development Environment	21
	4.2.2 Sensor Node Firmware Implementation	21
	4.2.3 CAN Communication Software	21
	4.2.4 Heartbeat Monitoring and Fault Detection	22
	4.2.5 Autonomous Task-Reallocation Software	23

	4.2.6 Wifi & Cloud Communication	23
	4.2.7 Supabase Database Implementation	24
	4.2.8 Webdashboard	24
5	Result & Discussion	28
6	Future Scope & Conclusion	30
	References	32

List of Figures

Figure no	List of figure	Page no
Figure 3.2.1	Block diagram of Architecture master system side	10
Figure 3.2.2	Block diagram of Architecture Node system side	11
Figure 3.3	PCB Design Voltage regulator	12
Figure 4.1.1	ESP32 Development board	15
Figure 4.1.2	SN65HVD230 CAN Module	16
Figure 4.1.3	DHT 11 sensor	17
Figure 4,1,4	Vibration sensor	17
Figure 4.1.5	IMU MPU6050 Sensor	18
Figure 4.1.6	Oled display	19
Figure 4.1.7	Relay 2 Channel Opto coupler Type	19
Figure 4.2.1	Web dashboard	25

ABBREVIATIONS

S. No.	Abbreviation	Full Form
1	CAN	Controller Area Network
2	ESP32	Espressif Systems 32-bit Microcontroller Series
3	IoT	Internet of Things
4	MCU	Microcontroller Unit
5	GPIO	General-Purpose Input/Output
6	DHT11	Digital Temperature and Humidity Sensor (model designation)
7	MPU6050	6-Axis Accelerometer and Gyroscope (device designation)
8	I²C	Inter-Integrated Circuit
9	SDA	Serial Data
10	SCL	Serial Clock
11	CANH	CAN High
12	CANL	CAN Low
13	TWAI	Two-Wire Automotive Interface
14	IDE	Integrated Development Environment
15	Wi-Fi	Wireless Networking Technology
16	HTTP	Hypertext Transfer Protocol
17	HTTPS	Hypertext Transfer Protocol Secure
18	API	Application Programming Interface
19	REST	Representational State Transfer
20	JSON	JavaScript Object Notation
21	UI	User Interface
22	DBMS	Database Management System
23	SQL	Structured Query Language
24	PCB	Printed Circuit Board
25	OLED	Organic Light-Emitting Diode

26	USB	Universal Serial Bus
27	IDE	Integrated Development Environment
28	RLS	Row Level Security
29	RTOS	Real-Time Operating System
30	PWM	Pulse-Width Modulation

Chapter-1

1.1 Introduction

The rapid development of embedded systems and the Internet of Things (IoT) has increased the demand for reliable, intelligent, and continuously operating sensor monitoring systems. Distributed embedded architectures enable multiple sensor nodes to collect data independently and exchange information through communication networks. However, failures in sensor nodes, communication links, or individual components can interrupt monitoring activities and reduce overall system availability. Therefore, implementing effective fault detection and autonomous recovery mechanisms is an important aspect of designing reliable embedded systems.

The Controller Area Network (CAN) is a message-based communication protocol widely used in automotive and industrial embedded applications. It supports communication between multiple electronic control units over a shared bus and provides mechanisms for detecting communication errors. In this project, CAN communication is employed to interconnect multiple ESP32-based sensor nodes with a central Master ESP32. The sensor nodes perform different monitoring functions using DHT11 temperature and humidity sensors, vibration sensors, and MPU6050 accelerometer and gyroscope modules.

The proposed system consists of four sensor nodes and one Master ESP32. Node 1 performs temperature and humidity monitoring, Node 2 monitors vibration, and Node 3 performs motion monitoring. Node 4 is designed as a universal backup node equipped with corresponding sensors to support the monitoring tasks of the primary nodes. The Master ESP32 monitors node availability through periodic heartbeat messages and identifies potential node failures using predefined timeout conditions. When a primary node fails, the Master initiates dynamic task reassignment to the appropriate backup sensor on Node 4, subject to the availability of the required sensor and successful task activation.

To improve system visibility and accessibility, the Master ESP32 transmits collected sensor readings, node health information, fault events, and task reassignment details to a cloud database through Wi-Fi and HTTP/HTTPS communication. Supabase is used for data storage and real-time updates, while a React.js web dashboard hosted on Vercel provides a graphical interface for monitoring sensor measurements, CAN communication status, node health, and backup recovery activities.

The main aim of this project is to integrate CAN-based distributed communication, heartbeat-based fault detection, autonomous backup task allocation, and cloud-based monitoring into a

unified embedded system. The proposed implementation is intended to demonstrate how a multi-sensor backup architecture can improve monitoring continuity and support fault-tolerant operation in distributed embedded applications.

1.2 Objective

1.2.1 Primary Objective

The primary objective of this project is to design and develop a CAN communication-based distributed sensor monitoring system using ESP32 microcontrollers, incorporating autonomous fault detection, dynamic task reallocation to a universal backup node, and cloud-based IoT dashboard monitoring. The system aims to improve monitoring continuity by detecting primary-node failures and transferring the affected sensor-monitoring task to an appropriate backup sensor on Node 4.

1.2.2 Specific Objectives

1. Development of a Distributed Sensor Network

To establish a distributed embedded architecture comprising a Master ESP32 and four sensor nodes interconnected through a Controller Area Network (CAN) bus. The network is designed to collect and transmit sensor measurements from DHT11 temperature and humidity sensors, vibration sensors, and MPU6050 accelerometer and gyroscope modules.

2. Implementation of Autonomous Fault Detection

To develop a node-health monitoring mechanism using periodic heartbeat messages and predefined timeout thresholds. The Master ESP32 evaluates node availability and identifies potential node failures or communication interruptions, enabling timely fault identification and status reporting.

3. Dynamic Task Reallocation to the Universal Backup Node

To design a dynamic task-reallocation mechanism in which Node 4 acts as a universal backup for the monitoring functions of Nodes 1, 2, and 3. When a primary node becomes unavailable, the Master ESP32 assigns the affected task to the corresponding backup sensor on Node 4, verifies activation, and updates the system's active sensor source.

4. Integration of Cloud-Based IoT Monitoring

To establish Wi-Fi connectivity between the Master ESP32 and the Supabase cloud database using HTTP/HTTPS communication. The system stores sensor measurements, node health information, CAN communication records, fault events, and task-reallocation details for remote monitoring and analysis.

5. Development of a Real-Time Web Dashboard

To develop a responsive React.js web dashboard hosted on Vercel and integrated with Supabase Realtime. The dashboard provides graphical visualization of sensor measurements, node operating status, communication activity, fault notifications, and backup activation events, allowing users to monitor system operation through a centralized interface.

6. Evaluation of System Reliability and Recovery Performance

To evaluate the proposed system under normal operating conditions and simulated primary-node failures. Performance assessment includes fault detection time, backup activation time, task-reallocation success rate, sensor-data continuity, and communication reliability to determine the effectiveness of the proposed fault-tolerant monitoring architecture.

1.2.3 Expected Outcome

The project is expected to demonstrate a distributed sensor network capable of detecting primary-node failures and automatically reallocating supported monitoring tasks to a universal backup node. Integration with the cloud database and web dashboard is intended to provide real-time visibility into sensor measurements, node health, and recovery activity.

The final system's effectiveness will be assessed through hardware experiments and measured performance results rather than assumed improvements.

CHAPTER-2

LITERATURE SURVEY

[1] Fault-Tolerant Time-Triggered Communication Using CAN

Reference: M. Short and M. J. Pont, "Fault-Tolerant Time-Triggered Communication Using CAN," IEEE Transactions on Industrial Informatics, vol. 3, no. 2, pp. 131–142, 2007.

The authors present a fault-tolerant communication mechanism for distributed embedded systems using the Controller Area Network (CAN). The proposed approach introduces redundancy management for replicated CAN channels to maintain synchronization among distributed nodes during communication failures. The study demonstrates the importance of reliable message exchange and coordinated operation in embedded control applications. The findings provide a foundation for developing dependable CAN-based sensor networks. In the proposed project, CAN communication connects multiple ESP32 nodes to the Master ESP32, which monitors node health and coordinates backup activation when a primary node becomes unavailable.

[2] A CAN-Based Application Level Error Detection and Fault Containment Protocol

Reference: J. R. Pimentel and J. Kaniarz, "A CAN-Based Application Level Error Detection and Fault Containment Protocol," IFAC Proceedings Volumes, vol. 37, no. 4, pp. 109–119, 2004.

The authors investigate application-level error detection and fault containment for distributed systems operating over a CAN network. The proposed protocol manages replicated components and determines how to switch from a failed primary component to an available replica. The study emphasizes fault identification, component isolation, and coordinated recovery in safety-critical applications. These concepts are relevant to the proposed system, where the Master ESP32 detects primary-node failures and assigns the affected monitoring task to the corresponding backup sensor on Node 4. The proposed implementation extends this concept to a multi-sensor embedded monitoring architecture.

[3] Dynamic Task Allocation in Multi-Hop Multimedia Wireless Sensor Networks with Low Mobility

Reference: Y. Jin, S. Vural, A. Gluhak, and K. Moessner, “Dynamic Task Allocation in Multi-Hop Multimedia Wireless Sensor Networks with Low Mobility,” Sensors, vol. 13, no. 10, pp. 13998–14028, 2013.

The authors propose a task-allocation framework that supports efficient in-network processing and resource sharing among wireless sensor nodes. The approach incorporates a rapid task-reallocation algorithm to recover from service disruptions caused by node or communication-link failures. The study highlights the importance of assigning tasks to available resources to improve network resilience. This work is relevant to the proposed project because the Master ESP32 reallocates monitoring tasks when a primary sensor node fails. Unlike the wireless sensor network approach, the proposed system uses CAN communication and a dedicated universal backup node equipped with DHT11, vibration, and MPU6050 sensors.

[4] Trust Dynamic Task Allocation Algorithm with Nash Equilibrium for Heterogeneous Wireless Sensor Network

Reference: W. Z. Guo, “Trust Dynamic Task Allocation Algorithm with Nash Equilibrium for Heterogeneous Wireless Sensor Network,” Security and Communication Networks, 2015.

The author investigates task allocation in heterogeneous wireless sensor networks, considering the limitations of sensor resources and shared communication channels. A trust-based dynamic allocation algorithm is developed using Nash equilibrium to support task distribution among available nodes. The study highlights the need to consider node capability and resource availability during task assignment. These principles are relevant to the proposed universal backup node, because the Master ESP32 must select an appropriate backup sensor and confirm its availability before reallocating a monitoring task. The proposed system applies a simpler rule-based allocation mechanism suitable for an ESP32-based prototype.

[5] Fault Detection Using Heartbeat Signal in the Real-Time Distributed Systems

Reference: Won-Sik Moon, “Fault Detection Using Heartbeat Signal in the Real-Time Distributed Systems,” Journal of Digital Industry Information Society, vol. 14, no. 3, pp. 39–44, 2018.

The author examines heartbeat-based fault detection in real-time distributed systems. The study discusses how periodic heartbeat signals can help identify failures and evaluates the relationship between heartbeat intervals and fault-detection propagation time. The findings indicate that shorter heartbeat cycles can reduce detection delay, although communication

overhead must also be considered. This approach directly supports the proposed system, in which the Master ESP32 monitors periodic heartbeat messages from the four sensor nodes. When a node exceeds the configured timeout, the Master initiates fault handling and backup-task reassignment.

[6] An In-Depth and Insightful Exploration of Failure Detection in Distributed Systems

Reference: B. Chaurasia, A. Verma, and P. Verma, "An In-Depth and Insightful Exploration of Failure Detection in Distributed Systems," Computer Networks, vol. 247, article 110432, 2024.

The authors present a systematic review of failure-detection techniques in distributed systems, examining node availability, communication failures, and fault-tolerance mechanisms. The paper discusses heartbeat-based monitoring, polling methods, and different approaches to failure detection and recovery. It also identifies challenges associated with selecting suitable detection thresholds and distinguishing actual failures from temporary delays. This review provides useful guidance for selecting heartbeat intervals and timeout thresholds in the proposed ESP32 network. The project applies these principles to detect unavailable sensor nodes and coordinate recovery through a universal backup node.

[7] Using the Heartbeat Failure Detector for Quiescent Reliable Communication and Consensus in Partitionable Networks

Reference: M. K. Aguilera, W. Chen, and S. Toueg, "Using the Heartbeat Failure Detector for Quiescent Reliable Communication and Consensus in Partitionable Networks," Theoretical Computer Science, vol. 220, no. 1, pp. 3–30, 1999.

The authors study heartbeat-based failure detection in distributed systems containing process failures and unreliable communication links. The research examines how heartbeat information can support reliable communication and coordination when some nodes become unavailable. The work establishes important theoretical foundations for detecting failed processes and maintaining communication in distributed environments. These principles are relevant to the proposed system's heartbeat mechanism, which allows the Master ESP32 to track node availability. The proposed implementation uses heartbeat timeouts as a practical trigger for assigning monitoring tasks to Node 4.

[8] A Permanent Fault-Tolerant Dynamic Task Allocation Approach for Network-on-Chip Based Multicore Systems

Reference: “A Permanent Fault-Tolerant Dynamic Task Allocation Approach for Network-on-Chip Based Multicore Systems,” Journal of Systems Architecture, vol. 97, pp. 287–303, 2019.

This study addresses dynamic task allocation in multicore systems where processing elements may experience permanent faults. The approach considers task mapping and scheduling to maintain application execution despite failures in individual processing units. The research highlights the importance of identifying available resources and reallocating tasks when a processing element becomes unavailable. These concepts are relevant to the proposed ESP32 system, where the Master ESP32 coordinates task reassignment after a primary sensor node fails. Although the original study focuses on network-on-chip multicore platforms, the proposed project adapts the general principle to sensor-task reassignment across separate microcontroller nodes.

[9] Distributed Reliable and Efficient Transmission Task Assignment for WSNs

Reference: “Distributed Reliable and Efficient Transmission Task Assignment for WSNs,” Sensors, vol. 19, no. 22, article 5028, 2019.

This paper investigates reliable and energy-efficient task assignment in wireless sensor networks. The proposed transmission-oriented task-allocation approach considers network-wide conditions and communication efficiency when distributing sensing and transmission responsibilities. The study emphasizes that task assignment influences sensing quality, transmission reliability, and resource utilization. These findings are useful when designing backup-task allocation for the proposed system. The Master ESP32 must consider sensor availability and CAN communication capacity before assigning Node 4 to replace a failed primary monitoring function.

[10] Design of an Infrastructure for Enabling Dynamic Fault Tolerance in Highly-Reliable Adaptive Distributed Embedded Systems Based on Switched Ethernet

Reference: A. Ballesteros et al., “An Infrastructure for Enabling Dynamic Fault Tolerance in Highly-Reliable Adaptive Distributed Embedded Systems Based on Switched Ethernet,” Sensors, vol. 22, no. 18, article 7099, 2022.

The authors investigate infrastructure for supporting dynamic fault tolerance in distributed embedded systems communicating over switched Ethernet. The research focuses on enabling systems to adapt their operation in response to failures while preserving important functions. The study provides a reference for designing coordinated fault-management mechanisms in

distributed architectures. The proposed project applies related concepts to a CAN-based sensor network, where the Master ESP32 identifies node failures and reallocates monitoring tasks to Node 4. A cloud-connected React.js dashboard with Supabase is also incorporated to visualize fault events, node status, and backup recovery activity.

CHAPTER-3

3.1 WORKING PRINCIPLE

The proposed system operates using a distributed embedded architecture that integrates multiple ESP32 sensor nodes, Controller Area Network (CAN) communication, heartbeat-based fault detection, autonomous task reallocation, and an Internet of Things (IoT) web dashboard. The main objective is to maintain continuous sensor monitoring by detecting the unavailability of a primary sensor node and automatically transferring its sensing task to a universal backup node. The system consists of one master ESP32, four sensor-node ESP32 boards, CAN transceiver modules, temperature and humidity sensors, vibration sensors, MPU6050 motion sensors, and a cloud-connected monitoring interface.

Initially, the master ESP32 and all sensor nodes are initialized with their respective hardware configurations, communication parameters, and sensor interfaces. Node 1 is assigned temperature and humidity monitoring using the DHT11 sensor, Node 2 performs vibration monitoring using a vibration sensor, and Node 3 measures acceleration and angular velocity using the MPU6050 sensor. Node 4 is configured as a universal backup node containing a DHT11 sensor, a vibration sensor, and an MPU6050 sensor. During normal operation, Nodes 1, 2, and 3 perform their designated sensing tasks, while Node 4 remains available to support recovery when a primary node becomes unavailable.

The sensor nodes periodically acquire measurements and transmit the collected information to the master ESP32 through the CAN bus. Each transmitted message contains an appropriate node identifier, message identifier, sensor data, and relevant status information. In addition to sensor measurements, the active nodes periodically transmit heartbeat messages to indicate their operational status. The master ESP32 receives these messages and maintains information about the latest communication received from each node. This arrangement enables the master to monitor sensor activity and communication availability without requiring a separate cloud connection for every sensor node.

Fault detection is implemented using a heartbeat timeout mechanism. The master ESP32 continuously checks whether the expected heartbeat messages are received within a predefined time interval. If a node fails to transmit its heartbeat within the configured timeout, the master identifies the node as potentially faulty and initiates the recovery procedure. A missing heartbeat may indicate a processor failure, power interruption, firmware malfunction, or

communication problem. Therefore, the system records the condition as a suspected node or communication failure until further diagnostic information becomes available.

After detecting a suspected failure, the master ESP32 identifies the sensing task associated with the affected primary node and selects Node 4 for task reassignment. The master transmits a task-assignment command through the CAN bus, specifying the required backup sensing function. For example, if Node 1 becomes unavailable, Node 4 activates its DHT11 sensor task to continue temperature and humidity monitoring. If Node 2 fails, the backup node activates its vibration-sensing task. Similarly, if Node 3 becomes unavailable, Node 4 activates its MPU6050 task to obtain acceleration and angular velocity measurements.

Upon receiving the task-assignment command, Node 4 validates the request, activates the corresponding sensor task, and begins collecting measurements. It then sends an acknowledgment to the master ESP32 to indicate whether the requested task has been activated. The master subsequently verifies the reception of valid sensor measurements from the backup node. Only after successful verification is the recovery operation recorded as confirmed. If the backup node fails to activate the requested task or valid measurements are not received within the specified interval, the master records the recovery failure and updates the system status accordingly. The initial implementation is intended to support recovery from one primary-node failure at a time, subject to the processing capacity and communication bandwidth of the backup node.

The master ESP32 also provides connectivity between the local CAN-based sensor network and the cloud monitoring platform. Using Wi-Fi, the master transmits selected sensor readings, node health information, fault events, and task-reassignment records to a Supabase cloud database through an HTTPS-based interface. The database stores these records for subsequent retrieval and monitoring. A React.js web dashboard retrieves the stored information and uses real-time database updates to present the current system condition. The dashboard displays sensor measurements, online and faulty node status, CAN communication activity, backup-node assignments, fault notifications, and recovery events.

When a previously unavailable primary node becomes operational again, the master can apply a predefined recovery policy to restore the original task assignment. Before transferring the task back, the master verifies the node's communication stability and sensor-data validity. The backup task is released only after the primary node's operation has been confirmed. Consequently, the proposed system integrates distributed sensing, heartbeat-based fault

detection, automatic task reassignment, recovery verification, and cloud-based visualization into a unified embedded monitoring architecture. The effectiveness of the system can be evaluated experimentally using fault-detection time, task-recovery time, sensor-data interruption, CAN communication reliability, and backup activation success rate.

3.2 System Design

The system design describes the hardware architecture, software architecture, communication interfaces, fault-detection mechanism, task-reallocation procedure, and cloud-monitoring framework of the proposed system. The system is developed using a distributed embedded architecture in which multiple ESP32 sensor nodes as shown figure 3.2.1communicate with a master ESP32 through a Controller Area Network (CAN) bus as shown figure 3.2.2.

The proposed architecture consists of one master node and four sensor nodes. Three sensor nodes perform dedicated sensing tasks, while the fourth node functions as a universal backup node. The master monitors node health using heartbeat messages, detects suspected failures using timeout parameters, and transfers the affected sensing task to the backup node. Sensor readings and system events are subsequently transmitted to a cloud database and displayed through a web dashboard.

The design is divided into hardware design, embedded software design, communication design, fault management, task reassignment, and IoT dashboard integration.

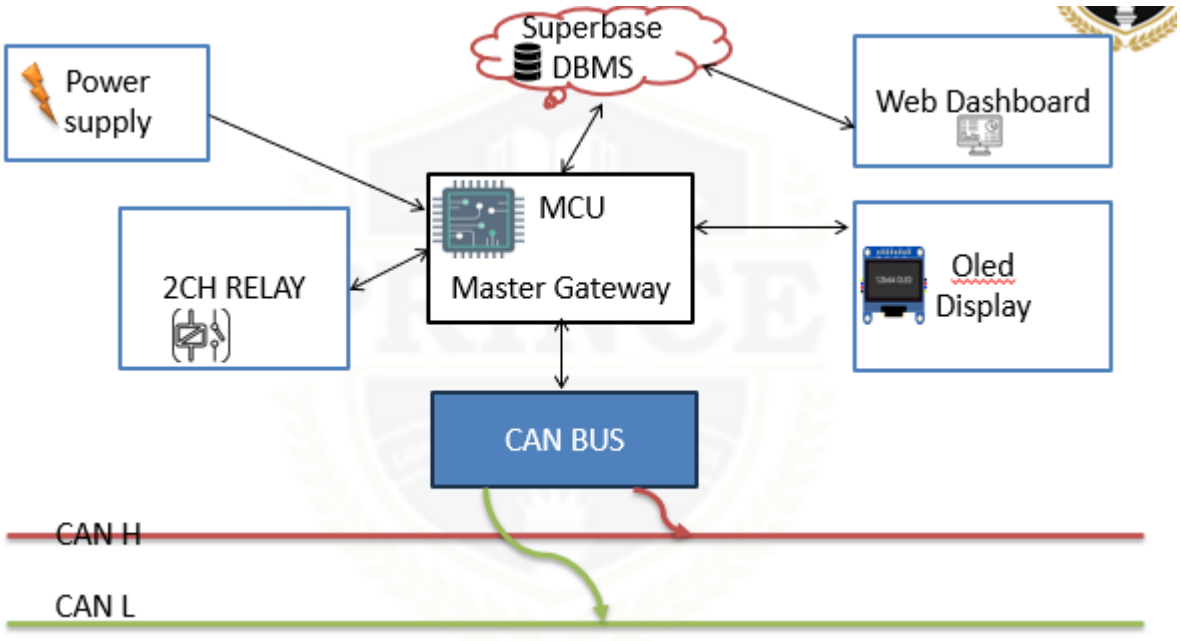


Figure 3.2.1 Block diagram of Architecture master system side

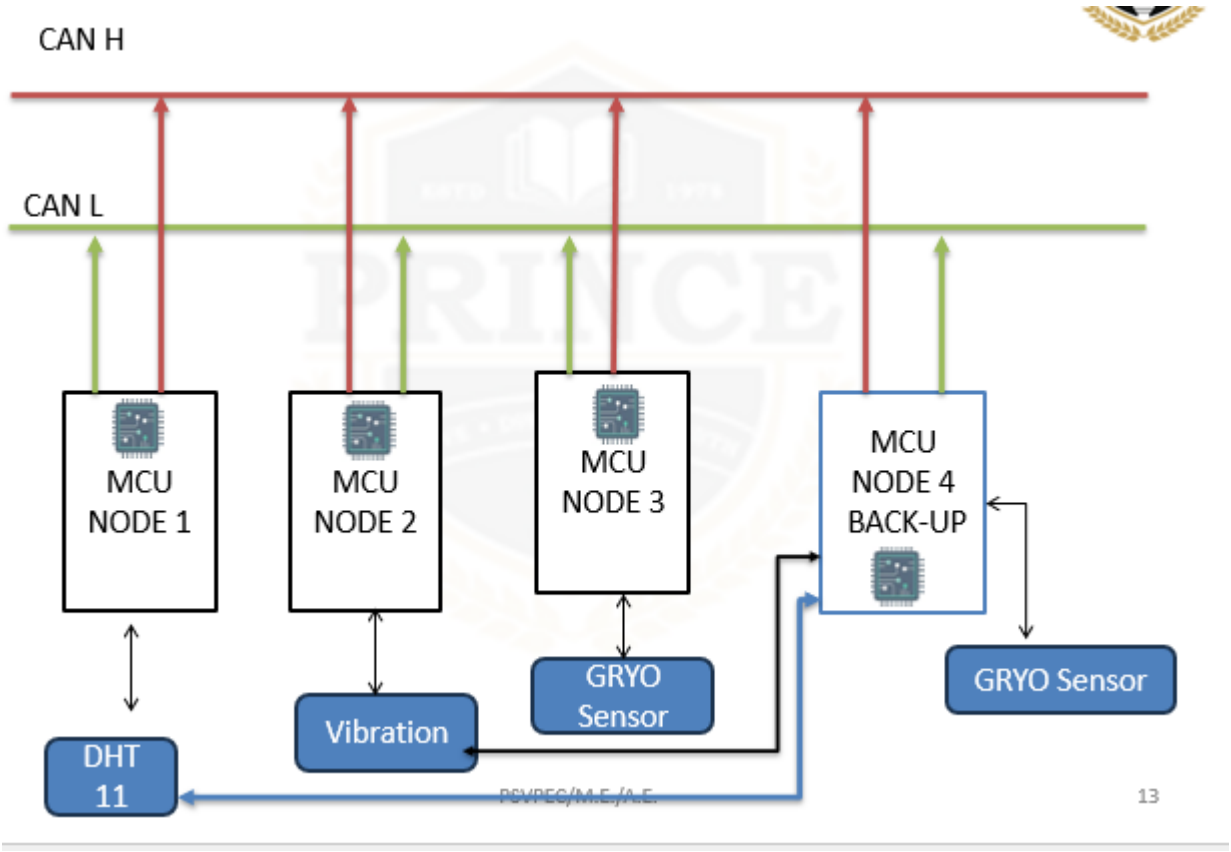


Figure 3.2.2 Block diagram of Architecture node system side

3.3 PCB Design Of Power Section

The designed voltage regulator circuit, as shown in the PCB layout, consists of multiple linear voltage regulator stages implemented using adjustable voltage regulator ICs (such as LM317). The circuit is designed to provide stable and regulated DC output voltages from a common input supply, ensuring reliable operation of connected electronic components is shown in Figure 3.3.1.

From the PCB structure, it can be observed that three identical regulator sections are arranged in parallel, each consisting of a voltage regulator IC mounted with a heat sink, resistors for voltage adjustment, capacitors for filtering, and LED indicators for output status. This modular design allows multiple output voltages to be generated simultaneously, which is useful in systems requiring different voltage levels.

Each regulator section follows a standard LM317 configuration. The input voltage is supplied to the regulator IC, and the output voltage is determined using a resistor network or potentiometer. The presence of blue trimmer potentiometers in the circuit indicates that the output voltage of each regulator can be adjusted precisely according to system requirements.

Capacitors are placed at both the input and output sides of the regulator to ensure voltage stability and reduce noise. The input capacitors help in filtering ripple from the DC source, while the output capacitors improve transient response and maintain a steady output voltage under varying load conditions.

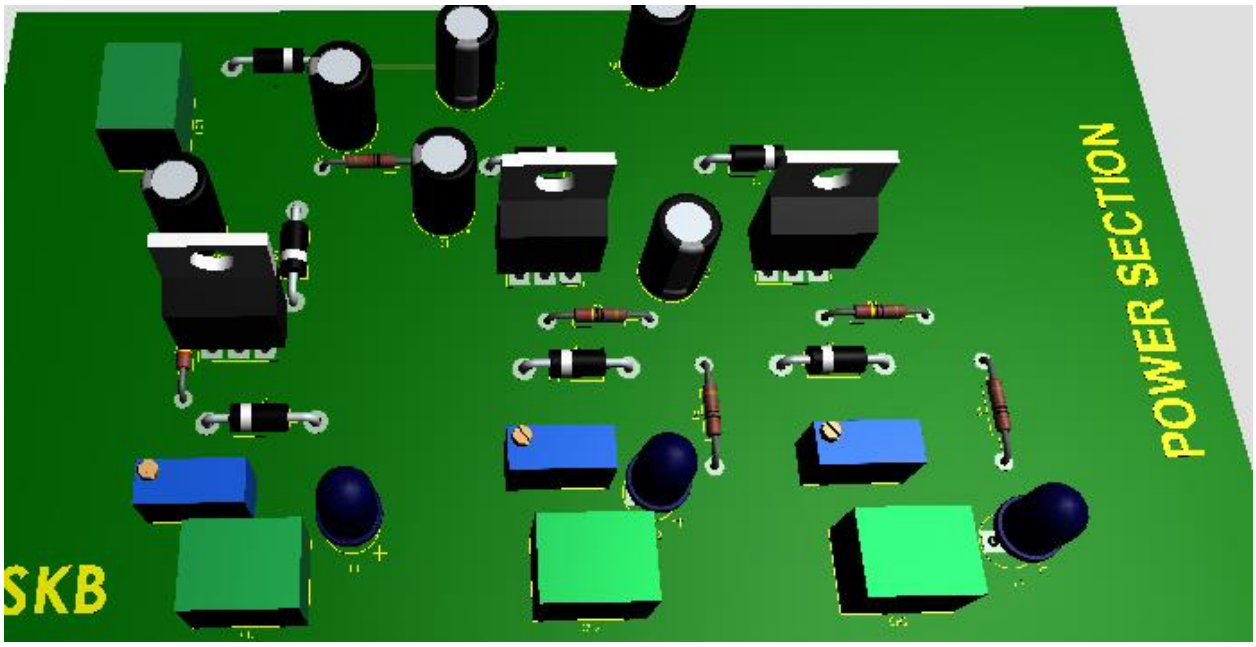


Figure 3.3 PCB Design Voltage regulator

LED indicators are included in each regulator section to visually indicate the presence of output voltage. When the regulator is functioning correctly, the corresponding LED glows, providing a quick and effective way to monitor the circuit status.

The use of heat sinks attached to the regulator ICs is an important design consideration. Since linear regulators like LM317 dissipate excess voltage as heat, proper thermal management is necessary to prevent overheating and ensure long-term reliability.

The PCB layout appears to be well-organized, with proper spacing between components, which helps in reducing thermal interference and improving maintainability. The modular arrangement also simplifies troubleshooting, as each regulator section can be tested independently.

One of the advantages of this design is its simplicity and reliability. However, linear regulators are less efficient compared to switching regulators, especially when there is a large difference between input and output voltage. This results in power loss in the form of heat.

Overall, the designed voltage regulator circuit provides a stable, adjustable, and user-friendly power supply solution. It is suitable for embedded system applications where multiple regulated voltages are required.

CHAPTER-4

HARDWARE & SOFTWARE IMPLEMENTATION

4.1 Hardware Implementation

The hardware implementation of the proposed system involves the integration of ESP32 microcontrollers, CAN transceiver modules, sensor modules, a master controller, and supporting electronic components. The system is designed to provide distributed sensor monitoring, automatic fault detection, and autonomous task reassignment through a universal backup node. The hardware architecture consists of one master ESP32 development board and four sensor-node ESP32 development boards connected through a shared Controller Area Network (CAN) bus.

The master ESP32 performs the primary coordination functions, including sensor-data reception, heartbeat monitoring, fault detection, backup-task assignment, and cloud communication. Node 1 is equipped with a DHT11 temperature and humidity sensor, Node 2 uses a vibration sensor, and Node 3 contains an MPU6050 accelerometer and gyroscope. Node 4 is designed as a universal backup node with all three sensor types. When a primary node becomes unavailable, the master instructs Node 4 to activate the corresponding sensing task and verifies the reception of valid sensor measurements before confirming recovery.

4.1.1 ESP32 Development Board

The ESP32 development board is used as the processing platform for the master controller and sensor nodes. It provides GPIO interfaces, serial communication peripherals, processing capability, and integrated Wi-Fi connectivity. A 38-pin ESP32 development board as shown figure 4.1.1 is used for the master, while four 30-pin ESP32 development boards are used for the sensor nodes.

The master ESP32 receives CAN messages from the sensor nodes and processes the information required for system coordination. It maintains the operational status of each node, detects heartbeat timeouts, and issues task-assignment commands when a primary node becomes unavailable. The master also connects to a Wi-Fi network to transmit selected sensor readings and system events to the cloud database.

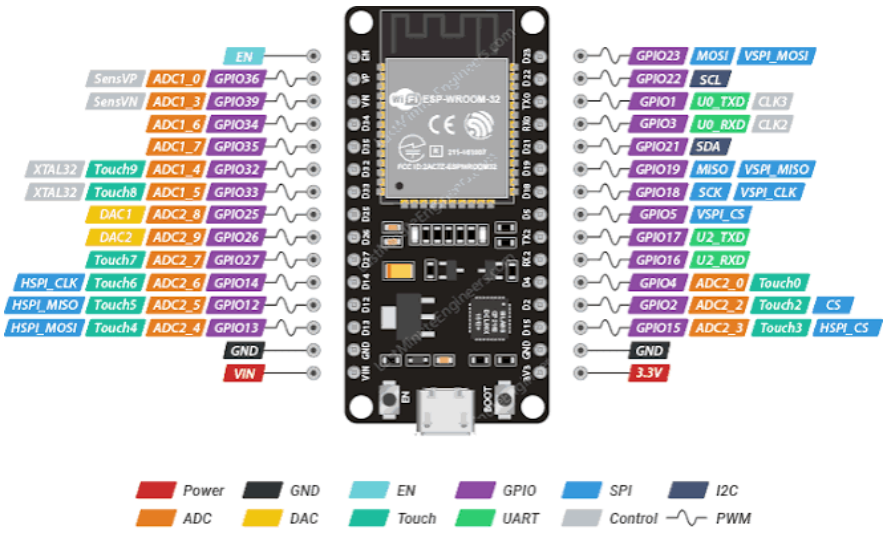


Figure 4.1.1 ESP 32 Dev Board

Each sensor-node ESP32 acquires measurements from its connected sensor, processes the data, and prepares CAN messages for transmission. Node 4 additionally executes the backup sensing task requested by the master. The firmware is developed using the Arduino IDE or a compatible embedded development environment. GPIO assignments are selected according to the actual board pinout and the interfaces required by the connected peripherals.

4.1.2 SN65HVD230 CAN Transceiver Module

The Controller Area Network provides the communication link between the master ESP32 and the four sensor nodes. Since the CAN controller interface and the physical CAN bus use different electrical signalling, each ESP32 requires a compatible CAN transceiver. Five transceiver modules are therefore required for the proposed five-board architecture.

The CAN transceiver converts the controller's transmit and receive signals into differential CANH and CANL signals. Sensor measurements, heartbeat messages, task-assignment commands, acknowledgments, and recovery-status information are transmitted through the shared bus. CAN message identifiers enable the receiving controller to distinguish between different message types and sources.



Figure 4.1.2 SN65HVD230 CAN Module

For a conventional high-speed CAN network, termination resistors are installed at the two physical ends of the bus. A typical arrangement uses one 120 Ω resistor at each end. The transceiver supply voltage and logic-level compatibility must be verified before connecting the module to the ESP32. Appropriate grounding, decoupling, and bus wiring are also necessary to reduce communication errors.

The selected ESP32 CAN interface must be compatible with the transceiver. If a module uses the SN65hvd230 controller as shown figure 4.1.2 , it communicates with the ESP32 through SPI and requires a compatible CAN transceiver. In contrast, an ESP32 with an integrated TWAI controller can interface with a suitable external CAN transceiver without requiring an SN65hvd230controller.

4.1.3 DHT11 Temperature and Humidity Sensor

The DHT11 is a digital environmental sensor used to measure temperature and relative humidity. It is connected to Node 1 for primary environmental monitoring. The ESP32 communicates with the sensor through a single-wire digital data interface and periodically retrieves the measured values.

After acquiring the measurements, Node 1 prepares a sensor-data message containing the relevant values and transmits it to the master through CAN. The master processes the received readings and makes them available for local or cloud-based monitoring.

A second DHT11 sensor as shown figure 4.1.3 is installed on Node 4 to support backup temperature and humidity monitoring. When the master detects the unavailability of Node 1, it assigns the corresponding sensing task to Node 4. The backup node activates its DHT11 reading

routine and begins transmitting measurements. Recovery is confirmed only after the master receives valid data from the backup task.



Figure 4.1.3 DHT 11 Sensor

The DHT11 is suitable for low-cost prototype development. Its measurement range, accuracy, response time, and sampling limitations must be considered when evaluating the quality of the acquired data.

4.1.4 Vibration Sensor Module

The vibration sensor is used to detect mechanical vibration or disturbance. One vibration sensor as shown figure 4.1.4 is connected to Node 2 for primary vibration monitoring, while a second sensor is installed on Node 4 for backup operation.



Figure 4.1.4 Vibration Sensor

Depending on the selected module, the sensor may provide a digital output indicating whether a vibration threshold has been crossed or an analogue output representing a varying signal. The ESP32 reads the available output and processes it according to the selected detection method.

The vibration status or measured signal is transmitted to the master ESP32 through CAN. The master can display the received information and record relevant events in the system log. If Node 2 becomes unavailable, the master commands Node 4 to activate its vibration-monitoring

routine. The backup node then collects vibration data and transmits it to the master for verification.

For reliable operation, the vibration sensor must be mounted securely and tested under known conditions. If the module provides only a threshold-based digital output, the system should describe its result as vibration detection rather than quantitative vibration measurement.

4.1.5 MPU6050 Accelerometer and Gyroscope

The MPU6050 is a motion-sensing device that integrates a three-axis accelerometer and a three-axis gyroscope. It measures acceleration along the X, Y, and Z axes and angular velocity around these axes. Communication between the MPU6050 and the ESP32 is established through the Inter-Integrated Circuit (I^2C) interface.

Node 3 uses the MPU6050 as shown figure 4.1.5 the primary motion sensor. The ESP32 periodically reads the sensor registers, obtains the acceleration and angular velocity values, and prepares the data for transmission through CAN. The master receives these measurements and makes them available for monitoring and analysis.

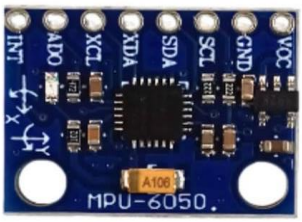


Figure 4.1.5 IMU MPU6050 Sensor Accelerometer and Gyroscope

A second MPU6050 is connected to Node 4 to provide backup motion monitoring. If Node 3 becomes unavailable, the master transmits a task-assignment command instructing Node 4 to activate its MPU6050 processing routine. The backup node starts acquiring motion measurements and sends them through CAN. The master verifies the received data before declaring that the sensing task has been restored.

Each MPU6050 is connected to the local I^2C bus of its respective ESP32 board. The SDA and SCL lines of separate sensor nodes are not connected together. Suitable pull-up resistors,

correct power connections, and valid I²C addressing must be ensured for reliable sensor communication.

4.1.6 OLED Display

An OLED display as shown figure 4.1.6 is included as an optional local monitoring interface for the master ESP32. It can display selected system information, including node availability, the latest fault status, the active backup task, and selected sensor measurements.



Figure 4.1.6 OLED 0.96 inch Display

The display typically communicates with the master using I²C. Its SDA and SCL lines are connected to the corresponding master GPIO pins, subject to verification of the actual development-board configuration. The display is useful during hardware testing because it allows the operator to observe local system status without relying entirely on the cloud dashboard.

The OLED is a monitoring component and does not perform the fault-detection or task-reallocation functions. These functions are executed by the master firmware.

4.1.7 Two-Channel Relay Module

A two-channel relay module as shown figure 4.1.7 may be included for auxiliary switching operations if the implemented system requires controlled switching of an external load or indicator. The ESP32 controls the relay inputs through suitable GPIO outputs.



Figure 4.1.7 Relay 2 Channel Opto Coupler type

The relay module is optional for the core proposed architecture because CAN communication, heartbeat monitoring, fault detection, and backup-task reassignment do not require relay operation. If included, the relay should be used only for a clearly defined control function. Its input voltage, logic compatibility, contact ratings, and isolation requirements must be verified before connection.

Relay contacts and relay-control logic should be treated as separate electrical sections. External loads must remain within the specified contact ratings, and appropriate protection must be provided for inductive loads.

4.1.8 Power Supply and Interconnection

A regulated power supply is required to operate the ESP32 development boards, sensors, CAN transceivers, and supporting modules. The supply arrangement must provide adequate current while maintaining each component within its permitted operating-voltage range.

The hardware connections include sensor interfaces, CAN communication lines, local I²C buses, power-distribution wiring, and ground connections. For a non-isolated prototype, the boards and transceivers generally require a common reference ground. CANH and CANL are connected to form the shared bus, with termination at the physical ends.

Before powering the complete system, the wiring should be inspected for incorrect polarity, short circuits, and incompatible signal levels. Each sensor module should first be tested individually. CAN communication should then be verified between the master and one sensor node before the remaining nodes are connected. Finally, the complete system should be tested for sensor acquisition, heartbeat transmission, fault detection, backup-task activation, and cloud data transfer.

4.2 SOFTWARE IMPLEMENTATION

The software implementation of the proposed system involves developing embedded firmware for the ESP32 master controller and sensor nodes, implementing CAN bus communication, configuring heartbeat-based fault detection, executing autonomous task reassignment, and integrating the embedded network with a cloud-based monitoring dashboard. The software is

divided into embedded firmware, communication management, fault-management logic, cloud communication, database services, and web application development.

The embedded firmware is developed using the Arduino IDE with appropriate libraries for sensor interfacing, CAN communication, Wi-Fi connectivity, and data processing. The master ESP32 coordinates the distributed sensor network, while the four sensor nodes execute their assigned sensing tasks. The cloud platform uses Supabase for data storage and real-time updates, and the web dashboard is developed using React.js, Vite, TypeScript, and Tailwind CSS.

4.2.1 Embedded Software Development Environment

The Arduino IDE is used to develop, compile, and upload firmware to the ESP32 development boards. It provides a programming environment based on C/C++ and supports libraries for interfacing with sensors and communication peripherals.

The ESP32 board support package provides the required compilation tools and hardware interfaces. Sensor libraries are used to acquire measurements from the DHT11 and MPU6050, while an appropriate vibration-sensor interface is implemented according to the selected module. The CAN communication software depends on the selected controller architecture. An ESP32 with an integrated TWAI controller can use a compatible TWAI driver and external CAN transceiver, whereas an MCP2515-based module requires an appropriate SPI-based CAN library.

The firmware is organized into functional modules to improve readability, debugging, and maintenance. Sensor acquisition, CAN message handling, heartbeat monitoring, task assignment, and cloud communication are implemented as separate logical components.

4.2.2 Sensor Node Firmware Implementation

Each sensor node runs firmware that initializes the ESP32 peripherals, configures the sensor interface, establishes CAN communication, and periodically transmits sensor readings and heartbeat messages.

Node 1 reads temperature and humidity values from the DHT11 sensor. Node 2 reads the vibration sensor output and determines the corresponding vibration status or measured signal. Node 3 reads acceleration and angular velocity from the MPU6050 through I²C. The acquired

measurements are validated and encoded into CAN data frames before transmission to the master ESP32.

The firmware also monitors incoming CAN messages. Node 4 contains additional control logic to receive task-assignment commands from the master and activate the corresponding backup sensor routine. After processing the command, it sends an acknowledgment indicating whether the requested task was accepted. It subsequently transmits the relevant sensor measurements to support recovery verification.

Heartbeat generation should operate independently of sensor-reading operations wherever possible. This helps maintain consistent health monitoring when sensor acquisition or data processing takes longer than expected.

4.2.3 CAN Communication Software

CAN communication software provides the data exchange mechanism between the master ESP32 and the four sensor nodes. Each node transmits sensor readings and heartbeat messages using predefined CAN identifiers. The master receives these messages and determines their source and purpose from the identifier and payload format.

The software defines separate message types for heartbeat transmission, sensor data, task-assignment commands, acknowledgments, and recovery-status information. Consistent message identifiers and payload definitions are required across all firmware modules.

When a sensor node transmits a measurement, the master identifies the corresponding node, validates the received frame, and processes the sensor data. When a fault is suspected, the master sends a task-assignment message to Node 4. The backup node receives the command, checks the requested task, and returns an acknowledgment.

The CAN communication module should handle transmission errors, invalid messages, and communication interruptions appropriately. Sequence numbers or timestamps can be used to identify stale or repeated application-level messages. The selected CAN controller, transceiver, bit rate, and message format must be configured consistently across all five ESP32 boards.

4.2.4 Heartbeat Monitoring and Fault Detection

Heartbeat monitoring is implemented in the master firmware to determine whether the sensor nodes remain responsive. Each active node periodically transmits a heartbeat message, and the master records the time at which the most recent valid heartbeat was received.

For the initial prototype, the heartbeat interval may be configured as 500 ms and the timeout as 2,000 ms. These values are initial design parameters and should be adjusted based on measured communication delays and scheduling behaviour.

The elapsed time since the last valid heartbeat is compared with the configured timeout. If the elapsed time exceeds the timeout, the master marks the node as potentially faulty and records the event.

The fault-detection sequence is as follows:

1. Receive heartbeat messages from the active nodes.
2. Update the last-received heartbeat time for each node.
3. Compare the elapsed time against the configured timeout.
4. Identify the node that has exceeded the timeout.
5. Record the suspected fault and initiate the recovery procedure.

A heartbeat timeout may indicate a node failure, a power interruption, a firmware problem, or a CAN communication fault. Consequently, the software should identify the condition as a suspected failure rather than assuming that the microcontroller itself has definitively failed.

4.2.5 Autonomous Task-Reallocation Software

The task-reallocation module enables the master ESP32 to assign a failed primary node's sensing task to Node 4. The module maintains a mapping between primary nodes and their corresponding sensor functions.

When a suspected failure is detected, the master determines the affected sensing task and sends a task-assignment command through the CAN bus. Node 4 interprets the command and activates the relevant sensor-processing routine.

The backup-task mapping is defined as follows:

Failed node	Reassigned task	Backup sensor
Node 1	Temperature and humidity monitoring	DHT11
Node 2	Vibration monitoring	Vibration sensor

Node 3	Motion and acceleration monitoring	MPU6050

After activating the requested task, Node 4 sends an acknowledgment to the master. The master then checks whether valid measurements are received from the backup node within a predefined verification interval. The recovery operation is marked successful only after this verification is completed.

If the command is rejected, the acknowledgment is missing, or valid sensor data is not received, the master records the unsuccessful recovery and reports the unresolved fault. The initial implementation is designed to recover from one primary-node failure at a time, with simultaneous failures requiring additional scheduling and resource-management logic.

4.2.6 Wi-Fi and Cloud Communication

The master ESP32 provides the connection between the local CAN network and the cloud monitoring platform. It uses the ESP32 Wi-Fi interface to connect to a configured wireless network and transmits selected sensor measurements and system events to Supabase through an HTTPS-based REST interface.

The cloud payload may include node identifiers, sensor values, measurement timestamps, heartbeat status, fault information, task assignments, and recovery results. The master should use a structured data format, such as JSON, to simplify communication with the cloud service.

The cloud communication module should handle Wi-Fi disconnections, HTTP errors, and unsuccessful database requests. Local fault detection and CAN-based task reassignment must continue even when cloud connectivity is unavailable. Data can be buffered locally where required and uploaded when connectivity is restored.

Secure authentication and appropriate database access policies must be implemented. Server certificate verification should remain enabled in the production system, and privileged database credentials must not be embedded in publicly accessible frontend code.

4.2.7 Supabase Database Implementation

Supabase is used as the cloud database platform for storing sensor readings and system events. The database organizes the information into separate tables according to the purpose of each record.

The proposed database tables include nodes, sensor_readings, can_messages, faults, task_assignments, and system_events. These tables store node status, timestamped

measurements, selected communication records, fault events, task assignments, and recovery information.

When the master uploads a new sensor reading, the cloud interface stores the measurement in the appropriate table. Similarly, a fault-detection event and the associated backup-task assignment are recorded so that the recovery sequence can be reviewed later.

Supabase Realtime can notify the web application when relevant database records change. Row-level security and suitable access policies should be configured to restrict unauthorized reading or modification of stored information.

4.2.8 Web Dashboard Implementation

The web dashboard is developed using React.js, Vite, TypeScript, Tailwind CSS, Lucide icons, Recharts, and Supabase client libraries. React components organize the interface into separate monitoring sections, while Vite supports development and production builds as shown figure 4.2.1 Web Dashboard Application

The dashboard provides the following functions:

- Displays the operational status of the master and sensor nodes.
- Presents temperature, humidity, vibration, and MPU6050 readings.
- Shows suspected node failures and active alerts.
- Displays CAN communication and selected system-event records.
- Indicates the sensing task assigned to Node 4.
- Presents backup activation and recovery-verification status.
- Maintains a history of detected faults and recovery events.

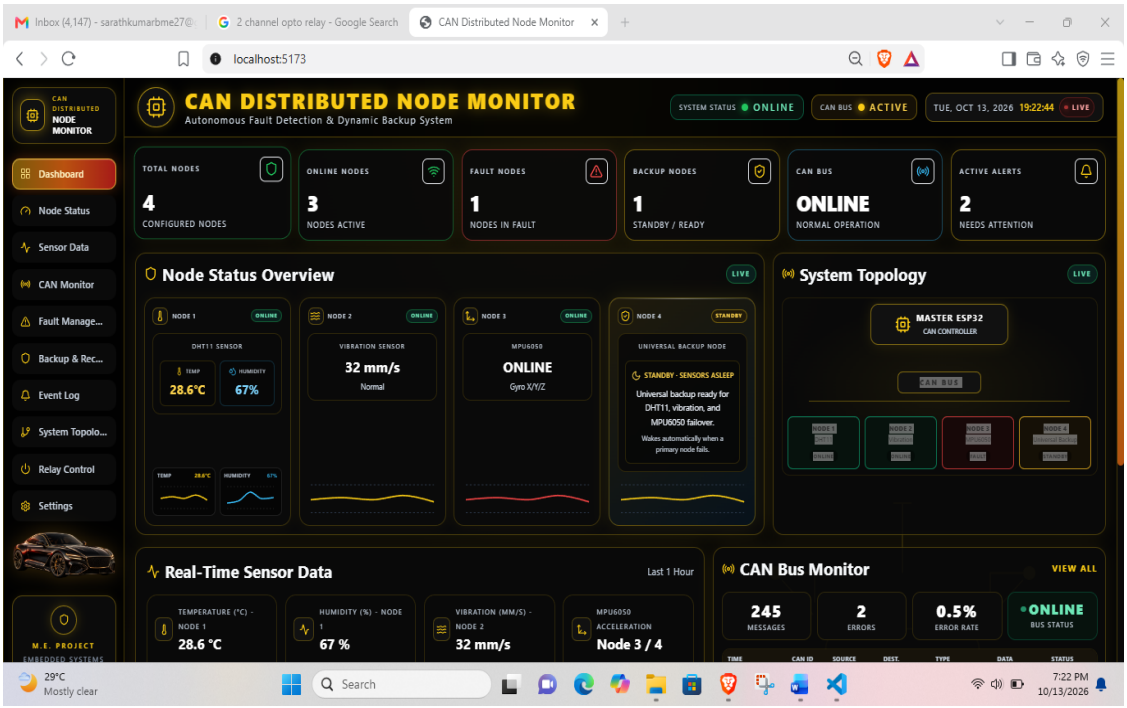


Figure 4.2.1 Web Dashboard

The application retrieves records from Supabase and updates the displayed information when new data becomes available. Recharts can be used to visualize sensor measurements over time, while status indicators distinguish online nodes, suspected faults, and active backup operations.

The dashboard is deployed on a hosting platform such as Vercel. Environment variables are used to configure the Supabase project URL and public client key. Database access is controlled through appropriate security policies rather than relying on frontend code to protect privileged credentials.

CHAPTER-5

RESULT & DISCUSSION

The proposed CAN-based multi-node ESP32 system is designed to provide distributed sensor monitoring, heartbeat-based fault detection, autonomous task reassignment, and cloud-based visualization. The system integrates one master ESP32 and four sensor nodes connected through a CAN bus. Three nodes perform dedicated sensing functions, while the fourth node acts as a universal backup containing a DHT11 temperature and humidity sensor, a vibration sensor, and an MPU6050 accelerometer and gyroscope.

The expected system operation begins with the acquisition of sensor measurements from the primary nodes. Each node transmits its sensor data and heartbeat messages to the master through the CAN communication network. The master processes the received information, monitors the operational status of each node, and identifies suspected failures when heartbeat messages are not received within the configured timeout interval. When a primary node becomes unavailable, the master assigns the corresponding sensing task to the universal backup node and verifies the reception of valid sensor measurements before confirming recovery.

During normal operation, Node 1 performs temperature and humidity monitoring, Node 2 detects vibration, and Node 3 measures acceleration and angular velocity. Node 4 remains available for backup operation. The master receives the sensor measurements and maintains the current status of the distributed network. This arrangement enables the system to monitor multiple physical parameters through a common communication infrastructure.

The fault-detection mechanism is based on periodic heartbeat messages. For an initial configuration with a heartbeat interval of 500 ms and a timeout of 2,000 ms, the master identifies a node as potentially faulty when no valid heartbeat is received within the specified timeout period. The actual detection time depends on the timing of the last received heartbeat, communication delays, and firmware scheduling. Therefore, the timeout value should be validated experimentally to avoid unnecessary fault indications and excessive detection delays.

The autonomous task-reallocation mechanism is intended to improve monitoring continuity during a primary-node failure. For example, when Node 2 becomes unavailable, the master sends a task-assignment command to Node 4 to activate its vibration sensor. Node 4 then begins collecting vibration data and transmits the measurements through CAN. The master checks the acknowledgment and verifies the incoming sensor data before recording the recovery as

successful. Similar procedures can be applied to temperature and humidity monitoring when Node 1 fails or motion monitoring when Node 3 fails.

The cloud-monitoring subsystem provides remote access to sensor measurements, node health information, fault records, and recovery events. The master ESP32 uploads selected information to the Supabase database through Wi-Fi and HTTPS. The React.js web dashboard retrieves the stored records and displays the system status using sensor-value indicators, graphical representations, fault notifications, and backup-task information. This integration allows the user to observe the distributed system without directly accessing the embedded hardware.

The effectiveness of the proposed system should be evaluated using experimental measurements, including successful CAN communication, heartbeat timeout detection, backup-task activation, recovery time, missed sensor samples, and cloud-data availability. Tests should be performed by interrupting each primary node individually under controlled conditions and checking whether Node 4 can perform the corresponding sensing task. Additional tests should examine CAN communication interruptions and temporary cloud disconnections to distinguish local network failures from Internet connectivity problems.

The results should be reported using actual measurements obtained during hardware testing. If the backup node successfully restores the affected sensing function, the observation can demonstrate the feasibility of the proposed task-reallocation mechanism. However, a successful prototype test alone does not establish complete fault tolerance or suitability for safety-critical applications. The system's limitations include temporary data interruption during recovery, possible false fault detection, backup-node resource constraints, and dependence on the availability of the master ESP32 and the CAN communication network.

Overall, the proposed architecture provides a practical approach to integrating distributed sensing, automatic fault detection, backup-task reassignment, and cloud-based monitoring using low-cost embedded hardware. The final experimental discussion should compare measured performance against the intended design objectives and identify opportunities for improving communication reliability, recovery speed, and system availability.

CHAPTER-6

FUTURE SCOPE & CONCLUSION

6.1 FUTURE SCOPE

The proposed CAN-Based Multi-Node ESP32 System for Fault Detection and Autonomous Task Reallocation to a Backup Node with IoT Dashboard Monitoring provides a foundation for improving the reliability of distributed embedded systems. The developed system integrates multiple ESP32 sensor nodes through the CAN communication network, implements heartbeat-based fault detection, and uses a universal backup node to take over the temperature and humidity, vibration, or motion-monitoring task when a primary node fails. The master controller manages node communication and recovery operations, while the IoT dashboard provides remote monitoring of sensor readings, node status, and fault events. In the future, the system can be enhanced by incorporating advanced fault diagnosis techniques to distinguish sensor failures from communication and hardware faults. Artificial intelligence and machine learning algorithms could be introduced to analyse historical sensor data and identify abnormal operating conditions for predictive maintenance. The backup mechanism could also be extended to support multiple simultaneous node failures through additional backup nodes and an improved task allocation algorithm. Implementing FreeRTOS would enable more structured real-time execution of sensor acquisition, CAN communication, fault detection, and cloud data transmission. Furthermore, the dashboard could be enhanced with historical data analysis, automated alerts, and detailed recovery reports, while stronger authentication and secure communication mechanisms could improve cybersecurity. The prototype could eventually be converted into a dedicated printed circuit board and evaluated for industrial equipment monitoring and other distributed sensing applications. Further experimental testing will be required to measure fault detection time, recovery time, communication reliability, power consumption, and backup success rate under different operating conditions. These developments can improve the system's scalability, robustness, and practical applicability.

6.2 Conclusion

The proposed project presents a CAN-based distributed embedded monitoring system that integrates ESP32 microcontrollers, multiple sensor modules, heartbeat-based fault detection, autonomous task reassignment, and cloud-based visualization. The architecture consists of a

master ESP32 and four sensor nodes, of which three perform dedicated sensing tasks and the fourth functions as a universal backup node.

The primary nodes monitor temperature and humidity, vibration, and motion parameters using the DHT11, vibration sensor, and MPU6050, respectively. The master ESP32 receives sensor measurements and periodic heartbeat messages through the CAN bus. When a primary node fails to provide a heartbeat within the configured timeout, the master identifies a suspected failure and assigns the corresponding sensing task to Node 4.

The universal backup node activates the required sensor-processing routine and transmits measurements to the master. The master verifies the reception of valid sensor data before recording the recovery as successful. This mechanism is designed to reduce monitoring interruption and demonstrate the feasibility of automatic task reassignment in a distributed embedded network.

The integration of Wi-Fi, HTTPS communication, Supabase, and a React.js dashboard provides remote access to sensor readings, node status, fault events, and backup recovery information. The modular architecture also offers opportunities for further development in advanced fault diagnosis, predictive maintenance, enhanced security, and real-time task scheduling.

In conclusion, the proposed system provides an academic prototype for studying fault detection and autonomous recovery in CAN-based distributed embedded systems. Its practical effectiveness must be established through hardware testing and quantitative evaluation of fault-detection time, recovery time, communication reliability, missed sensor samples, and backup-task activation success rate. The results of these evaluations can guide future improvements and support adaptation of the architecture to more demanding industrial and automotive monitoring applications.

References

[1] Aosong Electronics, “DHT11 Temperature and Humidity Sensor,” DHT11 Technical Documentation.

The DHT11 technical documentation describes the sensor's temperature and relative-humidity measurement functions, digital communication interface, and operating characteristics. It provides information required for connecting the sensor to a microcontroller and obtaining environmental measurements. The reference supports the implementation of primary temperature and humidity monitoring on Node 1 and backup monitoring on Node 4. The exact datasheet revision should be recorded according to the sensor module used.

[2] Arduino, “Arduino IDE Documentation,” Arduino Documentation, 2021.

This documentation provides information about the Arduino development environment, including writing, compiling, and uploading programs to microcontrollers. It explains sketch structure, library integration, serial communication, and debugging procedures. The reference is useful for developing and testing the ESP32 firmware, configuring the development environment, and implementing sensor acquisition and communication routines.

[3] Arduino, “Wire Library.”

This documentation explains the Wire library used for I²C communication between microcontrollers and peripheral devices. It describes the functions required to initialize the I²C interface and exchange data with connected devices. The reference is relevant to interfacing the MPU6050 modules and the optional OLED display with their respective ESP32 boards.

[4] Bosch, “CAN Specification Version 2.0,” Robert Bosch GmbH, 1991.

This specification describes the fundamental communication protocol used by Controller Area Network systems. It defines CAN message formats, identifiers, arbitration, frame transmission, and error-handling mechanisms. The reference provides the theoretical foundation for designing the communication structure between the ESP32 master and the distributed sensor nodes.

[5] Espressif Systems, “ESP32 Arduino Core Documentation,” Espressif Systems.

This documentation explains how to develop embedded applications for ESP32 boards using the Arduino framework. It covers board configuration, peripheral programming, supported libraries, and troubleshooting procedures. The reference supports the implementation of sensor

interfacing, GPIO configuration, serial debugging, and network communication in the proposed distributed embedded system.

[6] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems.

This datasheet describes the ESP32 architecture, processor capabilities, memory organization, GPIO interfaces, communication peripherals, and Wi-Fi functionality. It provides technical information required for selecting appropriate GPIO pins and configuring peripheral interfaces. The reference is particularly relevant to the master ESP32, sensor-node implementation, and Wi-Fi-based cloud connectivity.

[7] Espressif Systems, “ESP-IDF Programming Guide: Two-Wire Automotive Interface (TWAI),” Espressif Systems.

This documentation describes the ESP32 TWAI controller and its programming interface for CAN communication. It explains the configuration and operation of the controller for transmitting and receiving CAN frames. The reference is useful for implementing heartbeat messages, sensor-data transmission, task-assignment commands, and acknowledgments between the master and sensor nodes.

[8] Espressif Systems, “ESP-IDF Programming Guide: Wi-Fi,” Espressif Systems.

This guide explains the Wi-Fi networking capabilities of the ESP32 platform, including network initialization, connection management, and station-mode operation. It provides information required to establish wireless connectivity between the master ESP32 and the network. The reference supports the cloud-transmission subsystem, which forwards sensor readings, node status, and recovery events to Supabase.

[9] Microsoft, “TypeScript Handbook,” TypeScript Documentation.

This handbook explains TypeScript syntax, data types, interfaces, modules, and static type checking. It provides guidance for developing maintainable applications with structured data and reusable software components. The reference is useful for defining consistent data structures for sensor readings, node status, CAN events, and backup-task assignments in the web dashboard.

[10] React, “React Reference Documentation,” React Documentation.

This documentation describes the React component model and the programming interfaces used to build interactive user interfaces. It explains how components and state-management

features can be used to organize an application. The reference supports the development of the project dashboard, including node-status cards, sensor-data displays, fault notifications, and recovery-monitoring sections.

[11] Supabase, “Data REST API,” Supabase Documentation.

This documentation describes the REST interface generated from a Supabase PostgreSQL database. It explains how applications can perform database operations through HTTP requests and how access control is integrated with the database security model. The reference is particularly useful for transmitting sensor measurements, node-health information, fault records, and task-reassignment events from the master ESP32 to the cloud database.

[12] Supabase, “Getting Started with Realtime,” Supabase Documentation.

This guide explains how to develop applications that receive real-time updates using Supabase Realtime. It describes client configuration and the mechanisms available for receiving database changes and other events. The reference supports the implementation of the React.js dashboard, enabling changes in node status, sensor readings, and recovery information to be reflected in the monitoring interface.

[13] Supabase, “Row Level Security,” Supabase Documentation.

This documentation explains how PostgreSQL Row Level Security policies control access to individual database records. It describes the use of policies and permissions to restrict database operations according to the application's security requirements. The reference is relevant to protecting sensor data, fault records, and system information stored in Supabase and preventing unauthorized access through the web dashboard.

[14] Tailwind Labs, “Tailwind CSS Documentation,” Tailwind CSS.

This documentation explains the utility-class approach used to style web interfaces with reusable presentation rules. It describes techniques for creating layouts, managing spacing, applying colours, and building responsive components. The reference supports the development of the project's monitoring dashboard, allowing sensor readings, node-health indicators, fault alerts, and recovery information to be presented in a consistent and responsive interface.